CTF Solve Guide

Host reference — not for players. Explains the reasoning behind each flag, then the exact commands.

Flag 1 — ls & cat

This one just checks that players can move into a folder, list what's inside it, and read a file. Nothing is hidden and nothing needs special permissions.

```
cd flag1
ls
cat flag1.txt
```

Flag 2 — hidden files

The flag file's name starts with a dot, which makes it a hidden file on Linux. A plain `ls` won't show it, so players need `ls -la` to reveal all files, dotfiles included.

```
cd flag2
ls -la
cat .flag2.txt
```

Flag 3 — searching a large file

This file is far too long to scroll through and find the flag by eye. Instead of reading the whole thing, players should search it directly for the flag's format using grep.

```
cd flag3
grep "SC2{" flag3.txt
```

Flag 4 — extracting an archive

The flag is packed inside a zip archive, so it doesn't exist as a readable file until it's unzipped. Once extracted, the flag file can be read normally.

```
cd flag4
unzip archive.zip
ls
cat <the_extracted_file>
```

Flag 5 — running a program

This challenge has no puzzle beyond realizing you need to execute the binary sitting in the folder. It's setuid root, so running it as your normal user still lets it read the protected flag on your behalf and print it out.

```
cd flag5
./execute_me
```

> **What is setuid?** Setuid is a special permission bit on an executable. When it's set, the program runs with the permissions of whoever *owns* the file (here, root) instead of the permissions of whoever runs it. That's how a normal player can run this binary and still have it read a flag that their own account couldn't access directly.

Flag 6 — creating a specific file

`check_lain` is a setuid program that checks for a very specific file, in a very specific place, with very specific contents. Players just need to create a file named `lain` at `/tmp/lain` — no directory needed — and put the exact expected text inside it. Running the checker afterward confirms the file and reveals the flag.

```
cd flag6
touch /tmp/lain
echo "Wubba lubba dub dub" > /tmp/lain
./check_lain
```

Flag 7 — extract, then clean up

This one chains two skills together. First, players extract the provided tarball. The setuid helper `myname` then refuses to reveal anything until the leftover files are cleaned up: the extracted `/tmp/lain` file, and the original `bran.tar.gz` archive itself. Once both are gone, running the helper again prints the flag.

```
cd flag7
tar -xzvf bran.tar.gz
rm -rf /tmp/lain
rm bran.tar.gz
./myname
```

Flag 8 — switching users

The hint file in this folder only gives half the answer: a password, and a clue that the username starts with "blo". To get the actual full username, players need to check `/etc/passwd`, which lists every local account on the system. Once the username (`blossom`) is confirmed, players switch into that account with `su -` — the dashed form — and read the secret file waiting in blossom's home directory.

```
cd flag8
cat creds.txt
cat /etc/passwd          # look for the username starting with "blo"
su - blossom             # password: sunshine99
cat ~/secret.txt
exit
```

> **What's in /etc/passwd?** It's a plain text file listing every user account on the system — username, user ID, home directory, and default shell for each one. Unlike /etc/shadow, it's readable by everyone, which is exactly why it's the right place to look up a username.

> **su vs su -:** plain `su blossom` switches you to blossom's account but keeps your old shell environment and working directory — you can end up "as" blossom while still sitting in your own home folder. `su - blossom` (or `su -l blossom`) starts a fresh login shell instead: it loads blossom's own environment and drops you into blossom's home directory, just like if blossom had logged in directly. When in doubt, use the dash.

Flag 9 — the execute bit on a directory

The flag lives inside a subfolder called `gimmick`, but that folder is missing its execute permission. On a directory, the execute bit controls whether you're allowed to enter it — without it, `cd` fails even though the folder is visible. Adding the execute bit back fixes this.

```
cd flag9
ls -la
chmod +x gimmick
cd gimmick
cat flag9.txt
```

> **r, w, x on a directory, briefly:** read (r) lets you list what's inside; write (w) lets you create, delete, or rename things inside; execute (x) lets you actually enter the directory at all (cd into it). A directory without x is like a locked door you can still see the sign on.

Flag 10 — reading environment variables

The README points players toward a binary called `monster`, somewhere on the system (it's actually installed globally, so it can be run from anywhere). The binary asks for a secret value, which is stored in an environment variable rather than a file. Listing the environment and filtering for the right name reveals it; feeding that value back to the binary prints the flag.

```
cd flag10
cat README.txt
env | grep MONSTER_SECRET
monster
# paste the value shown when prompted
```

> **What's an environment variable?** It's a named value the shell keeps for the current session, which programs can read to change how they behave — things like $HOME or $PATH. It isn't stored in any file you can grep, which is exactly why this flag needs `env` instead of a text search.

Bonus — a guided, multi-step checker

This folder is deliberately full of noise: random junk files, a couple of fake flags planted to catch anyone blindly grepping for the flag format, and a file renamed to hide what it actually is. Rather than hunting file by file, just list everything recursively and look for the one file that doesn't belong with the clutter.

```
cd flagBonus
ls -R
```

Among everything that comes back, the file named `bin` inside `dir6` is the one that matters — it's a setuid-root checker that walks you through the rest of the puzzle step by step when you run it.

```
./dir6/bin
```

Heads up: the first time you run it, the checker prints a sentence to put in file1 — but that printed sentence is a decoy. The text it actually checks for is **"I love my life"**. Keep that in mind so you're not stuck chasing the wrong quote.

Step 1 — create file1 in dir1 with the real expected sentence:

```
echo "I love my life" > dir1/file1
./dir6/bin
```

Step 2 — move file1 into dir6, exactly as the checker asks:

```
mv dir1/file1 dir6/file1
./dir6/bin
```

Step 3 — create file2 in dir6, containing file1's new absolute path. `realpath` gives you that path without typing it by hand:

```
realpath dir6/file1
echo "$(realpath dir6/file1)" > dir6/file2
./dir6/bin
```

Step 4 — identify mirror's real type (dir10/mirror is a disguised file with no extension), then pass that type string back in as an argument:

```
file -b dir10/mirror
./dir6/bin 'paste the exact output here'
```

> i **Why single quotes, not double quotes, in that last command?** Single quotes tell the shell to treat everything inside them completely literally — no exceptions. Double quotes still let the shell interpret things like $, backticks, or escaped characters inside the string. The output of `file` can contain characters like these (commas, parentheses, sometimes $ or quotes), and the checker compares the argument byte-for-byte — so single quotes are the only way to guarantee what you pass in matches exactly what `file` printed.

Note for you as host: the flags sitting in dir3/flag.txt and dir4/flag.txt are intentional decoys, in the same SC2{...} format as the real flag — they exist to punish a blind `grep -r` instead of actually working through the checker.

Flag 11 — Umbra Mortis (cracking root's hash)

This is the privilege-escalation flag, and it's the longest one. After logging in as `almost_root`, the first move should always be to check who you are and what groups you belong to:

```
su almost_root       # password: peekaboo99
id
```

```
uid=1002(almost_root) gid=1003(almost_root) groups=1003(almost_root),42(shadow)
```

That output shows almost_root belongs to the `shadow` group — and members of the shadow group are allowed to read `/etc/shadow`, the file holding every account's password, including root's.

```
cat /etc/shadow      # copy root's hash line
exit
```

> i **What's a hash?** Linux never stores your actual password — it stores a hash of it instead: the output of a one-way scrambling function that's easy to compute in one direction but practically impossible to reverse. When you log in, the system hashes what you typed and checks whether it matches the stored hash. There are many different hashing algorithms (this workshop uses one called md5crypt) — you don't need to know how they work internally, just that they exist and that the right cracking tool needs to know which one it's dealing with.

Rather than guessing the hash type, identify it first — either with a command-line tool or a website:

```
nth --text 'root_hash_here'
```

or paste the hash into

https://hashes.com/en/tools/hash_identifier

Cracking a hash doesn't mean reversing it — it means guessing, fast. A cracking tool takes a huge list of candidate passwords (a wordlist), hashes each one with the identified algorithm, and compares the result to the stolen hash. A match means that candidate was the real password. This only works if the real password is in the wordlist, which is why a well-known list like rockyou.txt catches so many common or reused passwords.

Using hashcat:

```
hashcat -m 500 -a 0 'root_hash_here' rockyou.txt
```

> i **Arguments:** `-m 500` tells hashcat which hash mode to use (500 = md5crypt, from the identification step above); `-a 0` selects a straight dictionary attack (try each wordlist entry as-is); the hash and the wordlist file are the last two arguments.

Using John the Ripper instead:

```
john --format=md5crypt --wordlist=rockyou.txt hash.txt
```

> i **Arguments:** `--format` tells John which hash type to treat the input as; `--wordlist` points to the candidate password list; `hash.txt` is just a text file you've saved root's copied hash line into.

> i **Don't have these tools yet?** hashcat, John, and nth are all installable the normal way: `sudo apt install hashcat john nth`

> i **Spoiler for you as host:** The cracked password is `abuhamza1`.

Once the password is cracked:

```
su root
cat /root/flag11.txt
```